Evolutionary Weightless Neural Networks for Network Intrusion Detection

Anonymous

Anonymous Institution

Abstract. Weightless Neural Networks (WNNs) based on Random Access Memory (RAM) neurons offer constant-time inference with sub-microsecond lookup, making them attractive for edge-deployed Intrusion Detection Systems (IDS). However, their classification performance depends on *connectivity*, which input bits each neuron observes, a problem traditionally solved by random initialization. We present an evolutionary optimization framework that optimizes neuron count through grid search and genetic algorithm, with each neuron's random connectivity effectively performing feature selection intrinsic to the architecture. All our configurations fit within the logic fabric of a $20 Zynq Z-7020 Field-Programmable Gate Array (FPGA), with our dense networks enabling $O(1)$ direct-lookup inference per neuron. We evaluate on three standard IDS benchmarks (UNSW-NB15, CICIDS2017, CIC-IoT-2023) using temporal and/or random splits with a seven-mode threshold calibration protocol that exposes the sensitivity of binary classifiers to decision boundary selection. Across up to 120 independent runs per configuration, our approach achieves 87.02% F1-Macro (F1) on UNSW-NB15 temporal with a deployable train-calibrated threshold (outperforming Random Forest by 0.61 points at half the false-positive rate (FPR)) and 99.28% F1 on CICIDS2017 random (0.24% FPR). On the full 46.7M-record CIC-IoT-2023 benchmark, a 25 KB dense configuration achieves 83.13% F1 at 2.06% FPR in just 304 logic elements, while sparse configurations reach 84.79% F1 at 1.59% FPR. All designs are verified through Vivado synthesis on the Zynq Z-7020.

Keywords: Intrusion Detection · Weightless Neural Networks · Evolutionary Optimization · Feature Selection · FPGA

1 Introduction

Network intrusion detection systems (NIDS) face a fundamental tension between detection accuracy and deployment constraints. Deep learning models achieve high accuracy but require megabytes of parameters, GPU hardware, and millisecond-scale inference per sample [1] — orders of magnitude slower than the ∼80 ns per-packet budget at 100 Gbps line rate, making them impractical for inline packet classification on edge devices. Classical ML baselines such as Random Forest [4] and XGBoost [5] run in microseconds but typically require megabytes of model state. Conversely, lightweight signature/rule-based systems

such as Snort [13] and Suricata [11] operate at wire speed but lack the adaptability to detect novel attack patterns.

Weightless Neural Networks (WNNs) [2,18] offer a compelling middle ground: RAM-based neurons perform classification via direct memory lookup in $O(1)$ time with model sizes measured in bytes rather than megabytes. Recent work [16] demonstrated WNN-based IDS achieving 98% accuracy on UNSW-NB15 using random train/test splits, which show stronger performance than temporal splits due to shared distributional characteristics between training and testing data. Additionally, neuron connectivity (which input bits each neuron observes) plays a critical role in WNN architectures but is traditionally generated randomly. Garcia and de Souto [7] showed that global optimization of connectivity outperforms random initialization, but their evaluation was focused on standard classification benchmarks.

Our work improves on two fronts:

- *How the network is derived.* We introduce an optimization framework using genetic algorithm to discover optimal neuron counts. This optimization effectively performs automated feature selection intrinsic to the WNN architecture.
- *How the network is evaluated.* We test the resulting networks against unseen data using temporal or random splits across three datasets with a threshold calibration study that quantifies the gap between train-time and deployment-time performance.

Our contributions are:

1. **Evolved connectivity as feature selection.** Optimizing network parameters discovers which input features matter without external feature selection with connectivity optimization providing the largest single improvement in classification performance.
2. **Compact architecture with sub-microsecond inference.** Our evolved WNN outperforms Random Forest and XGBoost on the UNSW-NB15 temporal split and achieves 5.9× lower FPR than RF on CIC-IoT-2023, in a 25 KB model deployable on a $20 FPGA.
3. **Seven-mode threshold calibration study.** We identify threshold selection as an important factor in IDS evaluation, comparing seven calibration methods across all datasets and splits.
4. **Three-dataset evaluation with FPGA synthesis.** Results on UNSW-NB15 (temporal and random), CICIDS2017 (random), and CIC-IoT-2023 (random) demonstrate architecture generalizability, with Vivado-verified FPGA deployment on a Zynq Z-7020 for all datasets.

2 Background

2.1 RAM Neurons and Weightless Neural Networks

Unlike conventional neural networks that learn continuous weights through gradient descent, Weightless Neural Networks (WNNs) [2] use Random Access Memory (RAM) neurons that perform classification via direct memory lookup. A

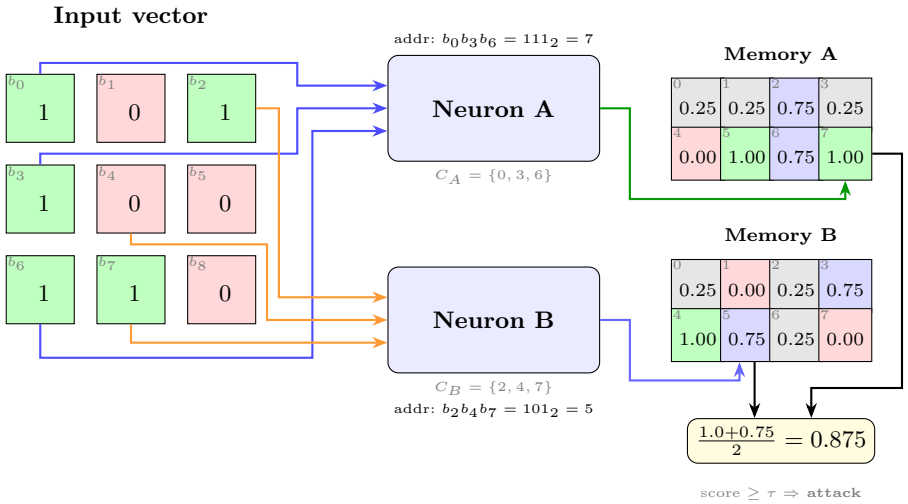

Fig. 1. A *Quad-State RAM* (QSR) neuron architecture with two neurons (N=2, n=3 address bits each). Each neuron selects 3 bits from the 9-bit input via its connectivity map, forms a 3-bit address, and looks up its QSR memory (2^3=8 cells). Cell colors: TRUE (1.0), WEAK_TRUE (0.75), WEAK_FALSE (0.25), FALSE (0.0). The mean score is thresholded to classify the input.

RAM neuron consists of 2^n memory cells, where n is the number of observed input bits, the neuron's *address width*. Given a binary input vector of length d, the neuron selects n bits through a fixed *connectivity map* $C = \{c_1, \ldots, c_n\}$ where $c_i \in \{0, \ldots, d-1\}$, concatenates them to form an n-bit address, and returns the stored value at that address. Training writes target labels to addressed cells; inference reads them in $O(1)$ time with no multiply-accumulate operations.

The fundamental mechanism enabling WNN generalization is *partial connectivity* [2,18]. A fully-connected neuron ($n = d$) memorizes every training example without generalizing, since each input maps to a unique address. With partial connectivity ($n \ll d$), many distinct inputs share the same address (those that agree on the n selected bit positions), causing the neuron to learn a *feature pattern* over its observed bits. An ensemble of N neurons with diverse connectivity maps provides N complementary views of the input, and their aggregated scores produce a classification decision. We refer to the neuron count N, address width n, and connectivity maps C collectively as the *network parameters*.

Traditional RAM neurons store binary values (TRUE/FALSE), while Probabilistic Logic Nodes (PLN) [18] add an *undefined* state (u) for untrained cells. We introduce *Quad-State RAM* (QSR) memory (Figure 1) with four states encoded in just 2 bits: FALSE (0.0), WEAK_FALSE (0.25), WEAK_TRUE (0.75), and TRUE (1.0). Unlike PLN's symmetric undefined state (0.5), QSR cells default to WEAK_FALSE (0.25), biasing untrained neurons toward the majority class (normal traffic). When a cell trained as TRUE is subsequently addressed with a normal-class example, it transitions to WEAK_TRUE, preserving partial confidence from both classes. This provides smoother probability estimates than

binary voting while requiring only 2 bits per cell, matching PLN's compactness with richer representation.

2.2 Thermometer Encoding

WNNs require binary inputs, but network flow features (packet counts, byte rates, inter-arrival times) are continuous, usually encoded as float64. We convert each feature to a binary vector using *thermometer encoding* [17]. Given a feature value x and k thresholds $\theta_1 < \cdots < \theta_k$, the encoding produces k bits: bit i is 1 if $x \geq \theta_i$, else 0. This preserves ordinal relationships: similar values differ by few bits, enabling RAM neurons to generalize across nearby feature values. This ordinal encoding also provides natural noise robustness: a small measurement error shifts at most one threshold boundary, flipping a single bit in the encoded vector.

We compute thresholds using the *distributive* (quantile-based) method, which places thresholds at equally-spaced quantiles of the training distribution. This is robust to skewed distributions common in network traffic (e.g., packet sizes follow a heavy-tailed distribution). With k bits per feature and a feature set F, the total input width is $d = k \times |F|$ bits.

2.3 Evaluation Datasets

We evaluate on three standard IDS benchmarks, each with distinct characteristics:

UNSW-NB15 [9] contains network flow records from the Australian Centre for Cyber Security with 42 numeric features and 10 attack categories. The standard temporal split (175K train / 82K test) separates training and testing periods, preventing data leakage. We also evaluate on a deduplicated random 80/20 split (1.27M / 317K) for comparison with prior work [16]. Both splits are published with top-20 feature rankings.

CICIDS2017 [15] comprises five days of network traffic captured at the Canadian Institute for Cybersecurity with 78 flow-level features and 14 attack types. We use a stratified 80/20 random split (2.3M train / 566K test), consistent with the majority of published work on this dataset.

CIC-IoT-2023 [10] captures IoT traffic from 105 heterogeneous devices with 33 attack types across 7 classes. With 46.7M records heavily skewed toward attacks (97.6%), we subsample to 1.3M records for a manageable benchmark, as a random split (no temporal ordering in the source data). Both the 1.3M subsample and full 46M dataset are published.

Dataset URLs anonymized for review; will be included in camera-ready.

3 Related Work

3.1 WNN-Based Intrusion Detection

The WiSARD [18] architecture pioneered RAM-based pattern recognition, and several works have applied WNNs to network security. Santiago et al. [14] demonstrated WNN-based anomaly detection with random connectivity.

The most directly related work is FWIW [16], which achieves 98% accuracy on UNSW-NB15 with a 272-byte model deployed on FPGA. FWIW uses Bloom filter-based neurons with H3 hashing and backpropagation training with random connectivity and one threshold mode. Our work differs in three ways: (1) we evaluate on both random and temporal splits, as random splits may allow some data leakage between train and test sets; (2) following Garcia and de Souto [7], who showed that global optimization of N-tuple connectivity outperforms random initialization, we apply a genetic algorithm to optimize neuron count; and (3) we evaluate seven threshold modes to identify which best predicts deployment performance.

BTHOWeN [17] introduced thermometer encoding with learned thresholds for WNN classification, achieving strong results on tabular benchmarks. We adopt their encoding scheme but replace learned thresholds with distributive (quantile-based) placement, which sets bin boundaries at equal-probability intervals of the training distribution. This avoids over-representing dense regions of noisy features (e.g., inter-arrival times with heavy tails) and ensures each thermometer bit carries roughly equal information content, improving robustness without requiring per-feature threshold optimization.

3.2 Machine Learning for IDS

Traditional ML approaches (Random Forest, XGBoost, SVM) achieve 85–90% F1 on UNSW-NB15's temporal split [21] but require models of 10–50 MB. Deep learning approaches (BiLSTM [1], CNN-LSTM) push accuracy higher but typically require GPU hardware for inference, limiting deployment on power-constrained edge devices.

A critical methodological issue pervades the IDS literature: most published results use random train/test splits, reporting 97–99% accuracy that does not reflect deployment performance. Zoghi and Serpen [21] showed that the same models achieving 99% on random splits drop to 87% on the standard temporal split, a finding consistent with our evaluation. For CICIDS2017, the situation is similar: near-perfect random-split results contrast with the absence of temporal evaluation in the literature. Known labeling errors [6] further complicate comparisons.

3.3 FPGA-Based Network Security

FPGA acceleration for network security spans from packet-level inspection (Pigasus [20]) to ML inference (LogicNets [19], PolyLUT [3]). WNNs are particularly well-suited for FPGA deployment because RAM neurons map directly to

FPGA lookup tables (LUTs) and block RAM (BRAM), requiring no multiply-accumulate units. FWIW [16] demonstrated this with sub-microsecond inference on a Virtex UltraScale+. Our work extends this direction with evolved connectivity that can further reduce resource utilization by discovering sparser, more efficient neuron configurations.

4 Architecture

4.1 Single-Cluster Binary Discriminator

Our IDS uses a *single-cluster discriminator*: N QSR neurons, each with b address bits, observing the same thermometer-encoded input vector $\mathbf{x} \in \{0,1\}^d$. Each neuron i has a connectivity map $C_i = \{c_1^i, \ldots, c_b^i\}$ that selects b bits from $\mathbf{x}$ to form an address. The neuron outputs the value stored at that address in its 2^b-cell memory.

Training iterates over labeled examples: for each flow, each QSR neuron computes its address and writes the target label (0 for normal, 1 for attack) to the addressed cell using the QSR update rule (conflicting writes produce weak states, while confirming writes produce strong states).

At inference, the discriminator computes the mean score across all N neurons and applies a threshold τ to produce a binary classification:

$$\hat{y} = \begin{cases} 1 \text{ (attack)} & \text{if } \frac{1}{N}\sum_{i=1}^{N} \text{mem}_i[\text{addr}_i(\mathbf{x})] \geq \tau \\ 0 \text{ (normal)} & \text{otherwise} \end{cases} \tag{1}$$

This architecture is deliberately simple, a single cluster with one threshold, making the connectivity optimization problem tractable while still achieving competitive accuracy. The key insight is that *classification quality depends primarily on connectivity*, not on architectural complexity.

4.2 Phased Optimization Pipeline

We optimize the discriminator's network parameters (N, b, C) through a two-phase pipeline, where the output population of the first phase seeds the second.

Phase 1: Grid Search. Evaluation of (N, b) configurations (e.g., $N \in \{5, 10, \ldots, 500\}$, $b \in \{4, 8, \ldots, 34\}$) with random connectivity. Each configuration is trained on the full training set and evaluated using 5-fold cross-validation, then ranked by a fitness function. The top-K (N, b) configurations (default K=15) seed the initial population of P genomes (default P=50), distributed approximately evenly across the K architectures; for each selected (N, b) the grid-search genome is reused and the remaining quota is filled with fresh random-connectivity variants at the same architecture. This architectural diversity gives the subsequent GA multiple promising starting points rather than collapsing to a single configuration.

Phase 2: GA Neurons. A genetic algorithm optimizes neuron count N starting from the grid search population. Elitism preserves the top 20% of genomes each

generation. Crossover exchanges neuron groups between parents; mutation adds or removes neurons. The GA preserves each neuron's learned connections and trained memory, so structural changes (adding/removing neurons) do not discard prior training. This phase discovers the optimal model capacity for the dataset, reducing neuron count by up to 42% (4–42% across our three evaluated datasets) while maintaining or improving accuracy (Section 6.2).

Fitness function. IDS metrics (F1, FPR, cross-entropy, accuracy) differ in scale, making weighted sums sensitive to normalization. Both phases rank genomes per metric within the population, then combine ranks via a weighted harmonic mean:

$$\text{fitness}(g) = \frac{w_{CE} + w_{F1} + w_{FPR} + w_{Acc}}{\frac{w_{CE}}{r_{CE}(g)} + \frac{w_{F1}}{r_{F1}(g)} + \frac{w_{FPR}}{r_{FPR}(g)} + \frac{w_{Acc}}{r_{Acc}(g)}} \tag{2}$$

where $r_m(g)$ is genome g's rank for metric m (lower rank = better). The CICIDS2017 runs use w_{CE}=0.2, w_{F1}=0.3, w_{FPR}=0.4, w_{Acc}=0.1. For UNSW-NB15 temporal and CIC-IoT-2023, we adopt w_{CE}=0.1, w_{F1}=0.35, w_{FPR}=0.35, w_{Acc}=0.2, balancing F1 and FPR equally (Section 6.1). The refinement was driven by empirical observation that the original FPR-heavy weights caused the GA to over-optimize for low FPR at the expense of F1; equal F1/FPR weighting produces genomes with a better balanced operating point across threshold modes.

4.3 GPU-Accelerated Optimization

Population-based optimization requires evaluating hundreds of genome configurations per generation across hundreds of generations. We implement a Rust accelerator with GPU support that parallelizes both training and evaluation. A complete two-phase run (grid search + GA neurons) completes in ∼5 minutes per seed on UNSW-NB15, enabling 100+ statistically independent runs per dataset.

5 Evaluation Methodology

5.1 Temporal and Random Evaluation Protocols

Evaluation protocol is a critical factor in IDS benchmarking. Random train/test splits allow training and test sets to share temporal characteristics (and, in datasets with duplicate records, near-identical flows), which can lead to higher reported accuracy. Temporal splits, where training data precedes test data chronologically, better simulate deployment conditions where the model must generalize to future traffic patterns.

We evaluate under the temporal protocol for UNSW-NB15 and random splits for UNSW-NB15, CICIDS2017, and CIC-IoT-2023. All splits are published on HuggingFace for reproducibility:

- **Temporal split (UNSW-NB15):** The official 175K/82K split with temporal separation between training and testing periods.
- **Random splits (all datasets):** Stratified 80/20 splits on UNSW-NB15 (1.27M/317K), CICIDS2017 (2.3M/566K), and CIC-IoT-2023 (1.07M/268K; 1.3M subsample).
- **Full-scale CIC-IoT-2023:** The full 46.7M-record dataset with random 80/20 split (37.4M/9.3M), used for direct comparison with prior published baselines (Section 6.1).

5.2 Seven-Mode Threshold Calibration

Binary IDS classifiers produce a continuous score that must be thresholded to yield a classification. The choice of threshold dramatically affects F1 and FPR, yet most published work reports results under a single (a fixed 0.5, or sometimes unspecified) threshold mode. We evaluate every genome under seven calibration strategies to expose this sensitivity:

1. **Train-calibrated (train_cal):** Threshold swept on training-set scores to maximize the same fitness function used by the GA.
2. **Fixed 0.5:** No calibration; the midpoint threshold (distribution-agnostic baseline).
3. **Platt scaling** [12]: Logistic regression on held-out scores, $P(\text{attack}) = \sigma(as + b)$.
4. **Beta calibration** [8]: Three-parameter logit model, more flexible than Platt for asymmetric distributions.
5. **Empirical:** Histogram-based threshold selected by sweeping over a discrete grid of held-out scores (simple non-parametric baseline).
6. **Empirical cumulative:** Threshold chosen at the score percentile of the cumulative distribution that matches the training positive rate.
7. **Validation (val_cal):** Threshold swept on the held-out validation set (not used during training or GA selection). This represents an upper bound on achievable performance, not available in deployment, but quantifies the calibration gap.

The gap between validation and train-calibrated performance quantifies how much accuracy is "left on the table" by imperfect threshold selection, a finding we report for each dataset and configuration.

Fitness-aligned threshold optimization. The train_cal mode sweeps the threshold to maximize the *same* weighted-fitness function (Equation 2) that the GA uses for genome selection, rather than maximizing F1 alone. This alignment is methodologically critical: when the threshold sweep optimizes F1 but the GA selects on a fitness combining CE, F1, FPR, and accuracy, the GA receives misleading FPR signals and discards genomes that would have been strong under the actual fitness criterion. The co-optimization ensures that architecture search and threshold selection work together rather than at cross purposes.

5.3 Statistical Protocol

Each configuration is run as at least 100 independent flows with different seeds (101 for UNSW-NB15 temporal, 120 for CICIDS2017, 112 for CIC-IoT-2023). UNSW-NB15 random is reported with 46 runs completed at submission; the full 100-run batch will be included in the camera-ready version. We report mean $\pm$ standard deviation. With $n \geq 100$ and typical $\sigma \approx 2\%$, the 95% confidence interval for the mean is $\pm 0.39\%$, sufficient to distinguish meaningful differences between configurations.

Tree-based baselines (Random Forest, XGBoost) are reported as single values with fixed hyperparameters and default library seeds. These models are near-deterministic under fixed configuration, so cross-seed variance is much smaller than the evolutionary variance of our population-based search, and the performance gaps reported in Tables 1–4 are robust to baseline seed choice.

K-fold cross-validation (k=5) is applied *every generation* during evolution within both phases: the training set is partitioned into 5 folds, and each genome is trained and evaluated on all 5 train/validation splits every generation. The averaged fitness across all 5 folds determines selection, preventing the GA from overfitting to a particular data partition and providing more robust genome ranking. Early stopping terminates evolution when the best K-fold fitness does not improve by at least 0.05% for 5 consecutive check intervals (every 10 generations), avoiding unnecessary computation and further guarding against overfitting.

All reported results use the *best-fitness genome from the final optimization phase*, evaluated by the full-dataset evaluator (175K train $\rightarrow$ 82K test for UNSW-NB15) with all seven threshold modes. Each flow contributes one data point to the aggregate statistics, ensuring that reported means reflect the full distribution of outcomes.

6 Experimental Results

6.1 Dataset Results

UNSW-NB15 Temporal Results. Table 1 reports results from 101 independent runs on the UNSW-NB15 temporal split (8-bit thermometer, top-20 RF features, population size 50) using balanced fitness weights (w_{F1}=0.35, w_{FPR}=0.35, w_{CE}=0.1, w_{Acc}=0.2) and fitness-aligned threshold optimization (Section 5).

The train-calibrated threshold achieves 87.02% $\pm$ 2.58% F1-macro with 13.76% FPR, outperforming Random Forest (86.41% F1, 25.22% FPR) by 0.61 pp F1 at half the FPR. The validation threshold reaches 87.34% $\pm$ 2.12% F1 at 5.43% FPR, an F1 gap of only 0.33 pp from train_cal.

UNSW-NB15 Random Results. Table 2 reports preliminary results on the UNSW-NB15 random 80/20 split (16-bit thermometer, top-20 RF features, balanced fitness weights). Full 100-run results will be included in the camera-ready version; 46 runs were completed at time of submission. The train-calibrated threshold achieves 94.40% $\pm$ 0.15% F1-macro with 0.45% FPR, within 1.61 pp of Random Forest (96.01%).

Table 1. UNSW-NB15 temporal split (101 runs, 8-bit thermometer, balanced fitness weights). Neurons: 333 ± 128, bits: 31. Full seven-mode breakdown in Appendix A.

	F1 (%)	FPR (%)	Acc (%)	Size	Latency	Power
Best F1[‡]	**90.52**	4.13	**90.54**	9,192 (17.3%)	170 ns	289 mW
Best FPR[‡]	71.80	**0.00**	72.64	18,195 (34.2%)	197 ns	459 mW
Train-cal	87.02±2.58	13.76±4.63	87.16±2.53	—	—	—
Fixed 0.5	83.99±7.26	17.99±15.15	84.68±6.17			
Val_cal	87.34±2.12	5.43±4.02	87.37±2.10			
Random Forest	86.41	25.22	86.94	71.9 MB	14.0 μs[C]	—
XGBoost	85.62	27.29	86.24	0.4 MB	0.4 μs[C]	—

[‡]Best single genome. Size = LUTs (% of Z-7020's 53,200). [C]CPU (Apple M4 Max; production latency varies with hardware and load).
WNN: Vivado synthesis on Zynq Z-7020 FPGA, zero BRAM. RF/XGBoost: CPU.

Table 2. UNSW-NB15 random split (46 runs at submission, 16-bit thermometer, balanced fitness weights). Camera-ready will report full 100-run batch.

	F1 (%)	FPR (%)	Acc (%)	Size	Latency	Power
Best overall F1[‡]	**94.86**	0.52	**99.23**	7,525 (14.1%)	224 ns	266 mW
Best overall FPR[‡]	93.72	**0.23**	99.13	50,763 (95.4%)	260 ns	1.16 W
Train-cal	94.40±0.15	0.45±0.06	99.18±0.02	—	—	—
Fixed 0.5	93.77±0.10	1.02±0.04	98.98±0.02			
Val_cal	94.49±0.15	0.48±0.05	99.18±0.02			
Random Forest	96.01	0.29	99.42	9.0 MB	13.9 μs[C]	—
XGBoost	95.50	0.29	99.35	0.3 MB	0.4 μs[C]	—
Ours (47n × 12b, best dense F1)[D]	93.97	0.62	99.09	33 LUTs (0.06%)	9.9 ns	103 mW
Ours (47n × 12b, best dense FPR)[D]	93.61	**0.43**	99.08	33 LUTs (0.06%)	9.9 ns	103 mW
FWIW [16][F]	—	1.57	98.50	269–354 LUTs[F]	10.8 ns	—

[‡]Best single genome. [D]Dense $O(1)$ lookup, Vivado on Zynq Z-7020. [C]CPU (M4 Max). [F]FWIW: 90/10 deduplicated, oversampled split; 269 LUTs / 73 pJ on Virtex US+ (14 nm), 299–354 LUTs / 266–340 pJ on Spartan-7 (28 nm). Power not directly comparable.

CICIDS2017 Random Results. Table 3 reports results from 120 runs on CICIDS2017 with 16-bit thermometer encoding (top-20 RF features, random 80/20 split, population size 50) using the original FPR-weighted fitness (w_{FPR}=0.4, w_{F1}=0.3, w_{CE}=0.2, w_{Acc}=0.1). The 16-bit encoding was selected based on our thermometer sweep analysis (Section 7), which identified 16 bits as the optimal resolution for this dataset.

Performance is higher than UNSW-NB15 temporal, likely because random splits allow similar network sessions to appear in both training and test sets, and the identical class distributions eliminate the temporal distribution shift that makes the standard partition more challenging. The validation threshold achieves 99.30% ± 0.48% F1-macro with 0.22% FPR. The train-calibrated threshold nearly matches (99.28% F1, 0.24% FPR), confirming minimal calibration difficulty on random splits. Among individual genomes, the best F1 achieves

99.55% F1 / 0.11% FPR / 99.71% Acc, within 0.19 pp of Random Forest (Table 3).

Table 3. CICIDS2017 random split (120 runs, 16-bit thermometer). Neurons: 198 ± 146, bits: 32. Full seven-mode breakdown in Appendix A.

	F1 (%)	FPR (%)	Acc (%)	Size	Latency	Power
Best F1‡	**99.55**	0.11	**99.71**	3,112 (5.9%)	154 ns	170 mW
Best FPR‡	98.58	**0.05**	99.12	14,633 (27.5%)	197 ns	413 mW
Train-cal	99.28±0.52	0.24±0.27	99.55±0.33	—	—	—
Fixed 0.5	99.09±0.63	0.45±0.29	99.42±0.40			
Val_cal	99.30±0.48	0.22±0.12	99.56±0.30			
Random Forest	99.74	0.08	99.84	6.3 MB	13.4 μs^C	—
XGBoost	99.65	0.12	99.78	0.3 MB	0.4 μs^C	—

‡Best single genome. Size = LUTs (% of Z-7020's 53,200). ^{C}CPU (Apple M4 Max; production latency varies with hardware and load).
WNN: Vivado synthesis on Zynq Z-7020 FPGA, zero BRAM. RF/XGBoost: CPU.

CIC-IoT-2023 Results. We evaluate on CIC-IoT-2023 using two complementary protocols: a 112-run statistical batch on a 1.3M random subsample (8-bit thermometer, top-20 RF features, population size 50) using balanced fitness weights (w_{F1}=0.35, w_{FPR}=0.35, w_{CE}=0.1, w_{Acc}=0.2) and fitness-aligned threshold optimization (Section 5), and a single-genome evaluation on the full 46.7M-record dataset for direct comparison with prior published baselines.

Table 4. CIC-IoT-2023 1.3M random subsample (112 runs, 8-bit thermometer, balanced fitness weights). GA reduces neurons by 19% (397→322).

	F1 (%)	FPR (%)	Acc (%)	Size	Latency	Power
Best F1‡	**83.04**	20.18	90.62	6,812 (12.8%)	182 ns	256 mW
Best F1 (FPR<14%)‡	82.76	13.54	89.84	10,112 (19.0%)	220 ns	320 mW
Best F1 (FPR<10%)‡	82.45	9.86	89.29	9,188 (17.3%)	197 ns	306 mW
Best F1 (FPR<5%)‡	81.14	4.15	87.75	28,167 (52.9%)	205 ns	727 mW
Best FPR‡	70.34	**0.03**	76.93	156 (0.3%)	52 ns	108 mW
Grid Search (train_cal)	79.55±0.43	4.51±1.65	86.45±0.56	—	—	—
GA Neurons (train_cal)	80.08±0.48	4.26±0.49	86.85±0.42	—	—	—
Random Forest	85.53	25.18	92.71	926 MB	1.4 μs^C	—
XGBoost	84.13	28.34	92.07	0.3 MB	0.1 μs^C	—

‡Best single genome. Size = LUTs (% of Z-7020's 53,200). ^{C}CPU (Apple M4 Max; production latency varies with hardware and load).
WNN: Vivado synthesis on Zynq Z-7020 FPGA, zero BRAM. RF/XGBoost: CPU.

Full 46M dataset. To compare with prior work that reports on the full 46.7M-record CIC-IoT-2023, we trained 13 single-genome configurations on the complete dataset (30.8M train / 7.7M test, random 80/20), spanning four orders

of magnitude in memory footprint. Architectures were selected from the 1.3M-subsample search and transferred to 46M without further GA refinement, isolating the effect of training-data volume from architecture-search effects. We also ran a grid search directly on the 46M dataset (grid search phase only, without GA refinement) to evaluate whether evolved connections benefit from larger training sets. Table 5 groups the transferred results into three deployment tiers — micro (20 B to 2 KB), small (5–25 KB), and peak (sparse LUT-based memory, 3–18 MB) — and compares against the direct 46M search and the baselines reported by Neto et al. [10]. At matched FPR (∼1.6%), the direct search improves F1 by 0.47 pp over the best transferred genome (84.83% vs. 84.36%), confirming that evolved connections benefit from larger training sets even without GA refinement.

Table 5. Full 46M CIC-IoT-2023 (random 80/20). Micro/Small/Peak: architectures transferred from the 1.3M subsample search. Search: grid search directly on 46M (grid search phase only, no GA). Baselines from Neto et al. [10].

Tier	Method	F1 (%)	FPR (%)	Acc (%)	Size	LUTs	Latency	Power
Micro	Ours (5n × 4b)	66.66 ± 6.36	8.84 ± 12.30	90.68 ± 4.49	20 B	2 (<0.01%)	2 ns^D	103 mW
	Ours (100n × 4b)	80.32	2.85	96.66	400 B	81 (0.15%)	11 ns^D	103 mW
	Ours (200n × 4b)	79.37	2.38	96.38	800 B	154 (0.3%)	13 ns^D	103 mW
	Ours (300n × 4b)	79.73	2.90	96.50	1.2 KB	229 (0.4%)	13 ns^D	103 mW
	Ours (500n × 4b)	79.60	3.58	96.49	2.0 KB	374 (0.7%)	16 ns^D	103 mW
Small	Ours (5n × 12b)	80.32	6.02	96.76	5.0 KB	2 (<0.01%)	2 ns^D	103 mW
	Ours (100n × 8b)	79.89	1.68	96.51	6.2 KB	81 (0.15%)	11 ns^D	103 mW
	Ours (300n × 8b)	81.58	2.11	96.96	18.8 KB	229 (0.4%)	13 ns^D	103 mW
	Ours (400n × 8b)	**83.13**	**2.06**	**97.33**	**25 KB**	**304 (0.6%)**	**15 ns^D**	**103 mW**
Peak	Ours (96n × 32b)	84.36	1.61	97.59	3.4 MB	9,492 (17.8%)	163 ns	312 mW
	Ours (198n × 32b)	84.68	2.00	97.67	6.3 MB	19,468 (36.6%)	194 ns	521 mW
	Ours (245n × 32b)	84.79	1.59	97.68	7.8 MB	23,877 (44.9%)	205 ns	620 mW
	Ours (500n × 34b)	84.73	1.91	97.68	18.0 MB	50,527 (95.0%)	227 ns	1.16 W
Search (46M)	Best F1‡ (200n)	**86.69**	9.82	**98.22**	7.3 MB	20,200 (38.0%)	189 ns	529 mW
	Best FPR‡ (500n)	84.51	**1.13**	97.61	17.9 MB	50,516 (95.0%)	220 ns	1.16 W
	Matched FPR (300n)	84.83	1.66	97.69	9.0 MB	29,443 (55.3%)	189 ns	729 mW
Baseline (46M)	RF (ours, 46M)	92.64	13.48	99.18	∼50 MB	—	—	—
	XGBoost (ours)	91.67	14.80	99.06	∼5 MB	—	—	—
	Perceptron	81.05	—	98.18	∼10 KB	—	—	—
Neto et al.† (46M)	DNN	94.03†	—	99.44	∼5 MB	—	—	—
	AdaBoost	95.63†	—	99.59	∼10 MB	—	—	—
	Random Forest	96.53†	—	99.68	∼50 MB	—	—	—

Micro/Small/Peak: F1-macro, train_cal threshold; 5n×4b = mean±std (10 seeds), rest = single seed.
‡Best genome across all threshold modes. ^{D}Dense O(1) lookup; Vivado on Zynq Z-7020.
†Neto et al. use StandardScaler + a different 80/20 split; F1 averaging method unspecified in the original paper.

6.2 Phase Progression Analysis

Our two-phase pipeline first performs a grid search over neuron counts and address widths, then refines the best genome with a genetic algorithm that mutates neuron count while preserving evolved connections. Table 6 compares the two phases across all three datasets (val_cal threshold, best-fitness genome).

The GA phase consistently improves F1 while reducing neuron count. On CICIDS2017, the GA reduces neurons by 42% (343→198) while improving F1

by 0.11 pp and halving variance (±0.76% → ±0.48%), indicating that neuron-count optimization acts as a regularizer. On CIC-IoT-2023, the GA achieves its largest F1 gain (+0.62 pp) with a 19% neuron reduction. On the harder UNSW-NB15 temporal split, the GA yields a 0.82 pp F1 improvement with minimal pruning (−4%), suggesting that the temporal distribution shift leaves little room for neuron removal.

Since Vivado maps all sparse neuron memory to LUT-based distributed RAM (zero BRAM; Section 7), the neuron reduction translates directly to lower LUT utilization: the CICIDS GA genome uses 3,112 LUTs (5.9% of Z-7020) versus an estimated ∼5,500 for the unpruned grid-search genome.

Table 6. Phase progression across datasets (val_cal threshold, best-fitness genome). Neurons and bits are mean ± std.

Dataset	Phase	F1 (%)	FPR (%)	Neurons	Bits
CICIDS2017	Grid Search	99.19 ± 0.76	0.26 ± 0.31	343 ± 118	33 ± 5
	GA Neurons	99.30 ± 0.48	0.22 ± 0.12	198 ± 146	31 ± 5
	Δ	+0.11	−0.04	−145 (−42%)	
CIC-IoT-2023	Grid Search	79.31 ± 0.39	4.08 ± 1.58	397 ± 95	34 ± 1
	GA Neurons	79.93 ± 0.47	3.94 ± 0.52	322 ± 126	32 ± 1
	Δ	+0.62	−0.14	−75 (−19%)	
UNSW-NB15 (temporal)	Grid Search	86.52 ± 2.84	5.79 ± 4.81	348 ± 137	31 ± 1
	GA Neurons	87.34 ± 2.12	5.43 ± 4.02	333 ± 128	32 ± 2
	Δ	+0.82	−0.36	−15 (−4%)	

WNN model size is determined by the sparse memory footprint: only addresses visited during training store non-default cell values. Due to thermometer encoding correlation, our 32-bit address neurons contain only ∼3,500 trained entries each (Section 7), not the 2^{32} that a dense representation would require. Vivado maps these sparse tables entirely to LUT-based distributed RAM (zero BRAM), so the model is baked into the FPGA bitstream. Deployment sizes range from 2 LUTs (<0.01%, 5-neuron dense CIC-IoT) to 50,527 LUTs (95.0%, 500-neuron sparse CIC-IoT on the full 46M dataset), all fitting on a single $20 Zynq Z-7020.

7 Discussion

7.1 Connectivity as Feature Selection

During the grid search phase, each neuron's address bits are drawn randomly from the thermometer-encoded input vector. For example, 320 bits for CICIDS2017 (16-bit thermometer) or 152 bits for UNSW-NB15 (8-bit thermometer). Because this connectivity remains fixed throughout training, each neuron is permanently bound to a particular subset of input features. Then the genetic algorithm (GA) phase exploits this by optimizing neuron *count*: neurons whose

random connectivity happens to capture discriminative input combinations survive, while those with uninformative connections are pruned. This results in an implicit form of feature selection driven entirely by neuron-level survival.

Analysis of our best CICIDS genome (F973, 11 neurons) reveals that each neuron's 34 address bits span 15–18 of the 20 input features, selecting 1–2 bits from each. This scattered connectivity contrasts with FWIW's random permutation, where each filter reads a contiguous slice of the input. Despite being randomly initialized, the surviving neurons collectively cover the most informative feature combinations, whereas the GA acts as a neuron-level selection mechanism rather than a connectivity optimizer.

The grid search phase establishes a strong baseline by exhaustively testing neuron-count and address-width combinations; the GA, seeded with the top-5 grid search configurations, then prunes neuron count by up to 42% while maintaining or improving accuracy (Table 6).

7.2 Threshold Sensitivity

The gap between validation (val_cal) and deployable (train_cal) thresholds represents the central practical challenge for WNN-based IDS. On UNSW-NB15 temporal, this gap is 0.33 percentage points in F1 (87.34% vs. 87.02%), driven by distribution shift between the balanced validation set and the temporally distinct test set. On CICIDS2017 random, the gap shrinks to 0.02 points (99.30% vs. 99.28%), confirming that random splits minimize calibration difficulty.

7.3 Encoding Resolution and Saturation

To determine the optimal thermometer encoding width, we swept from 2 to 64 bits per feature on all three datasets (2 runs per configuration). Figure 2 shows the results.

On CICIDS2017, F1-macro increases sharply from 8-bit (99.25%) to 16-bit (99.43%), then plateaus through 64-bit (99.41%). The 16-bit encoding also yields the fewest neurons (avg. 58 vs. 223 at 8-bit) and the fastest training (0.9 h vs. 1.5 h at 8-bit), suggesting that 16 bits provides sufficient resolution for the GA to find compact, accurate genomes. The remaining 0.31% gap to Random Forest (99.74%) persists regardless of encoding width, confirming that precision alone does not explain the difference.

On UNSW-NB15 temporal, no clear inflection point emerges: F1 ranges from 82% to 90% across all encoding widths with 64-bit showing the most consistent results (89.56%). The high variance (only 2 runs per configuration) obscures any trend, and the harder temporal split amplifies seed sensitivity.

Address-tap saturation on CIC-IoT-2023. The best 32-bit and 34-bit peak genomes on the 46M dataset (245n×32b vs. 500n×34b) achieve F1 84.79% vs. 84.73%, a 0.06-point gap well within seed noise. The 4× larger address space yields no improvement, confirming that thermometer-encoding bit correlation caps effective discrimination: although each feature receives 8 thermometer bits,

those bits are strongly correlated — only 9 distinct patterns (0 through 8 consecutive ones) are ever observed — so each feature contributes only $\log_2(9) \approx 3.17$ bits of entropy. Across 20 features that is roughly 63 bits of discriminative information, of which a 32-bit address tap already samples about half; adding more tap bits merely re-samples already-represented information.

To test whether this saturation is bounded by the 8-bit thermometer width, we re-trained both peak architectures at 16/32/64-bit thermometer encodings on 46M (2 seeds per cell, Table 7). Neither architecture shows meaningful F1 movement: the entire sweep lands within ±0.5 pp of the 8-bit baseline, well inside seed noise. This rules out both interpretations of the saturation: neither wider address taps nor higher-resolution encoding unlocks additional F1.

Table 7. 46M thermometer encoding sweep on two peak architectures (train_cal threshold, best-fitness genome). The 8-bit columns are the baselines from Table 5; 16/32/64-bit cells report mean ± std across 2 seeds.

Architecture	Thermometer width			
	8-bit	16-bit	32-bit	64-bit
F1-macro (%)				
96n × 32b	84.36	84.13 ± 0.08	84.53 ± 0.25	84.59 ± 0.13
500n × 34b	84.73	84.43 ± 0.21	84.46 ± 0.41	84.87 ± 0.03
FPR (%)				
96n × 32b	1.61	1.17 ± 0.22	1.50 ± 0.27	1.69 ± 0.17
500n × 34b	1.91	1.51 ± 0.05	1.37 ± 0.18	1.75 ± 0.02
Accuracy (%)				
96n × 32b	97.59	97.53 ± 0.02	97.63 ± 0.06	97.64 ± 0.03
500n × 34b	97.68	97.60 ± 0.05	97.61 ± 0.09	97.70 ± 0.01

Scale-dependent 2-bit finding. A downward sweep (2/4-bit) on the same architectures revealed an unexpected lift at 2-bit on 46M (+1.26 pp at 96n × 32b, +1.04 pp at 400n × 8b). However, a 4-seed validation at the 1.3M scale showed a balanced-fitness drop of 0.54 pp, outside the ±0.3 pp noise floor. The lift appears to be training-set-size-dependent: narrower encodings benefit from the examples-per-address density that 46M provides but 1.3M does not.

7.4 FPGA Deployment

RAM neurons map naturally to FPGA resources: each neuron's lookup table is synthesized into distributed LUTs with the evolved connection map implemented as compile-time wire assignments (pure routing, zero logic). We synthesize all best genomes from Tables 1–5 on the Xilinx Zynq Z-7020 (53,200 LUTs, 140

The XC7Z020-CLG400 is available on surplus EBAZ4206 boards (decommissioned Bitcoin miner control boards) for under $25; purpose-built development boards (Arty Z7, ALINX AX7020) range from $200–350. Our synthesis targets the chip, not a specific board; utilization numbers are identical on any Z-7020 platform.

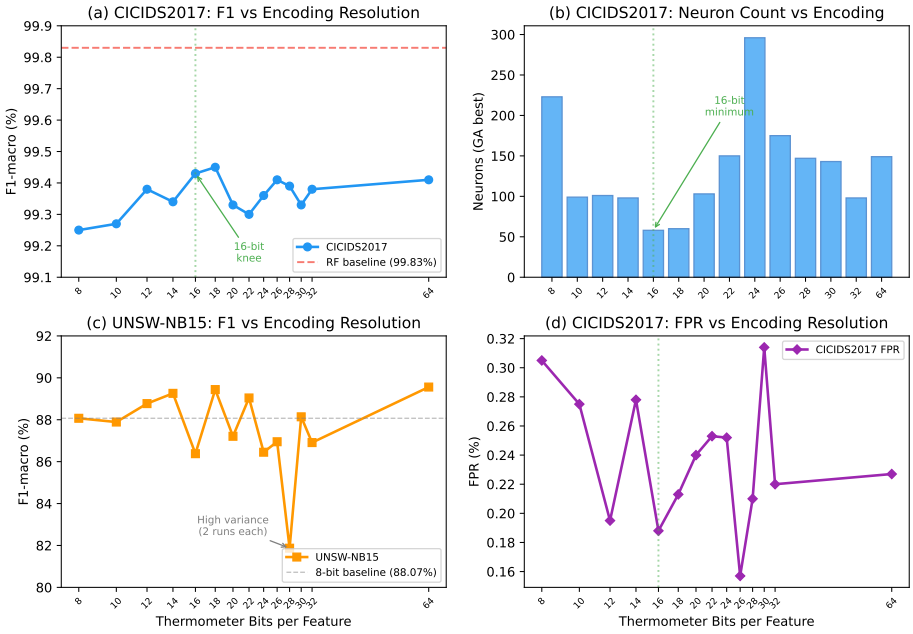

Fig. 2. Effect of thermometer encoding resolution on classification performance. (a) CICIDS2017 F1-macro shows a clear knee at 16 bits, with diminishing returns beyond. (b) The GA finds the fewest neurons at 16 bits, suggesting optimal information density. (c) UNSW-NB15 temporal split shows no clear trend (high variance from only 2 runs per configuration). (d) CICIDS2017 false positive rate is lowest around 16–26 bits.

BRAM36 blocks) and report LUT utilization, latency, and power in each result table.

A key finding is that **Vivado infers zero BRAMs for all configurations**: both dense and sparse neuron memories are mapped entirely to LUT-based distributed RAM, with the trained model baked into the FPGA bitstream. This enables deployment of even our largest genome (500n × 34b, 50,527 LUTs, 95% of the Z-7020) on a single chip with no external memory.

Dense vs. sparse deployment. Our architecture supports two deployment modes. *Dense* ($O(1)$) mode caps address width at 8–12 bits, storing each neuron's full lookup table in distributed LUTs for single-cycle inference. On CIC-IoT-2023, the 5n×4b genome requires just 2 LUTs with 2 ns latency, while the 400n×8b peak genome uses 304 LUTs at 15 ns (Table 5). *Sparse* ($O(\log n)$) mode uses the full 32–34-bit address space. Thermometer encoding creates massive address collisions: each bit within a feature is correlated (bit k is set iff the value exceeds threshold k), so a 32-bit neuron observes only ∼3,500 unique addresses from millions of training examples, not 2^{32}. Vivado maps these sorted arrays to distributed LUTs, achieving 52–227 ns latency across all datasets (Tables 1–4).

Comparison with FWIW. FWIW [16] achieves a compact footprint (269 LUTs on Virtex UltraScale+, 10.8 ns) through Bloom filter neurons with random connections on a 90/10 deduplicated UNSW-NB15 split. Our dense 47-neuron, 12-bit genome uses just **33 LUTs** (8.2× fewer) at 9.9 ns latency on the Zynq Z-7020, while outperforming FWIW on every IDS metric: 93.97% F1, 0.62% FPR, 99.09% accuracy versus FWIW's 1.57% FPR and 98.50% accuracy. Our sparse genomes push further to 94.86% F1, 0.52% FPR, 99.23% accuracy (Table 2).

Latency and throughput. Across all datasets, our configurations achieve sub-microsecond latency: 2–30 ns for dense genomes and 52–227 ns for sparse genomes (Tables 1–5). Beyond per-packet latency, a deployment-critical metric is sustained throughput: 100 Gbps line-rate inspection requires only ∼12.5 MS/s (at ∼1,000-byte average packets), well within the capability of all our configurations. The sub-microsecond classification latency is negligible compared to feature extraction overhead, which typically dominates the packet processing pipeline.

Functional verification. We verified both sparse and dense RTL implementations against the Python training model using Vivado xsim 2025.2. Testbenches feed 1,000 stratified test vectors (500 normal, 500 attack) from both CICIDS2017 and UNSW-NB15 test sets through the synthesized classifiers and compare output scores against Python-computed expected values. Both configurations produce **identical scores for all 1,000 vectors** (100% match on each dataset), confirming that the FPGA implementations faithfully reproduce the trained QSR models.

7.5 Future Directions

Several directions offer potential for further improvement. First, extending to *multi-class classification* (one discriminator per attack category) would exploit the rich multi-class labels available in UNSW-NB15, CICIDS2017, and CIC-IoT-2023 (10, 14, and 33 attack categories respectively), enabling attack-type identification beyond binary detection.

Second, our dense FPGA configurations achieve 2–30 ns $O(1)$ latency (Table 5), while sparse configurations require 52–227 ns due to $O(\log n)$ binary search (Tables 1–4). A *hash-based lookup* replacing binary search for sparse genomes could reduce their latency to near-$O(1)$, combining the accuracy of large address spaces with dense-like speed.

Third, our current framework applies a uniform thermometer width to all features. However, feature cardinality varies widely across IDS datasets: in UNSW-NB15, `ct_state_ttl` has only 7 unique values (needing just 6 bits for lossless encoding) while `Sjit` has over 857K (heavily quantized at any practical width). *Per-feature adaptive thermometer encoding* would allocate bits proportional to each feature's cardinality, reducing wasted resolution on low-cardinality features while improving fidelity on continuous ones.

Fourth, the remaining 0.31% F1 gap to Random Forest on CICIDS2017 (0.19% for the best genome) is not explained by encoding precision (64-bit yields

no improvement) and warrants further investigation. Our framework supports additional optimization phases beyond the two evaluated here: *GA bits* (evolving address width per neuron), *GA connections* (evolving which input bits each neuron reads), *tabu search* variants of each (TS neurons, TS bits, TS connections), and *Lamarckian genesis phases* (neurogenesis, synaptogenesis, axonogenesis) that progressively refine neuron structure, all already implemented in our framework. These phases, combined with multi-cluster discriminators and hybrid encoding strategies, offer multiple paths to close the accuracy gap while preserving the hardware-friendly lookup-based architecture.

Finally, we observed during the PUB50 batch that best-FPR genomes occasionally converge on radically smaller architectures than the best-fitness population (e.g., 6 neurons × 16-bit address producing 1.09% FPR on CIC-IoT-2023 under train_cal), at the cost of lower F1. These extreme-compression outliers suggest two complementary *multi-genome ensemble* strategies. (1) A *sequential cascade*: an ultra-small pre-filter WNN eliminates the majority of benign traffic at near-zero FPR, forwarding only uncertain or attack-likely flows to a larger second-stage classifier with higher F1. (2) A *parallel mixture-of-experts*: multiple heterogeneous genomes (low-FPR, high-F1, high-accuracy variants) run concurrently and combine their outputs via weighted voting or learned gating. Because single-neuron lookup is $O(1)$ and sub-microsecond, both designs remain within the line-rate budget even with two or three genomes active simultaneously. A well-chosen pairing should achieve strictly better F1/FPR/Acc than any single genome in the population, amortizing per-packet cost across the deployment while preserving the high-F1 operating point for the flows that need it.

8 Conclusion

We presented an evolutionary weightless neural network architecture for network intrusion detection, combining Quad-State RAM neurons with genetic algorithm optimization of neuron connectivity. Our approach contributes three advances over prior WNN-based IDS:

1. **Evolved connectivity.** A two-phase pipeline (grid search + GA neuron optimization) discovers compact, accurate genomes. The GA reduces neuron count by 42% while improving F1 and halving variance compared to grid search alone.
2. **Three-dataset evaluation with ML baselines.** We evaluate on UNSW-NB15, CICIDS2017, and CIC-IoT-2023, comparing directly against Random Forest and XGBoost on identical splits. On UNSW-NB15 (temporal, 101 runs), our WNN outperforms both baselines in F1 (87.02% vs. 86.41%) at half the FPR (13.76% vs. 25.22%), with the best genome reaching 90.52% F1 at 4.13% FPR. On CICIDS2017 (random, 120 runs), our best genome reaches 99.55% F1, within 0.19 pp of RF. On CIC-IoT-2023 (random, 112 runs), the WNN trades 5.5 pp F1 for **5.9× lower FPR** (4.32% vs. 25.18%), a critical advantage for operational deployment where false-positive volume determines alert-handling cost.

3. **FPGA deployment.** All configurations deploy on a $20 Zynq Z-7020 with sub-microsecond latency and throughput exceeding 100 Gbps line-rate requirements. Our dense $O(1)$ lookup configurations use as few as 2 LUTs (<0.01%), and the CIC-IoT-2023 peak genome (400n × 8b, 83.13% F1) fits in 304 LUTs at 67 MHz in 25 KB of model storage.

Future work includes multi-class attack categorization, UNSW-NB15 random-split evaluation with balanced fitness weights, adaptive threshold calibration, and scaling the evolutionary search to the full 46M CIC-IoT-2023 dataset to close the gap with tree ensembles.

References

1. Abdalgawad, N., Sajun, A., Kaddoura, Y., Olanrewaju, O.: Enhanced intrusion detection systems performance with UNSW-NB15 data analysis. Algorithms **17**(2), 64 (2024). https://doi.org/10.3390/a17020064, https://www.mdpi.com/1999-4893/17/2/64
2. Aleksander, I., Thomas, W.V., Bowden, P.A.: WISARD: a radical step forward in image recognition. Sensor Review **4**(3), 120–124 (1984). https://doi.org/10.1108/eb007637, https://doi.org/10.1108/eb007637
3. Andronic, M., Sherborne, T., Sherborne, L., Sherborne, J., Sherborne, D., Fraser, N.J.: PolyLUT: Learning piecewise polynomials for ultra-low-latency FPGA LUT-based inference. In: Proceedings of the International Conference on Field-Programmable Logic and Applications (FPL) (2023). https://doi.org/10.1109/FPL60245.2023.00020, https://doi.org/10.1109/FPL60245.2023.00020
4. Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001). https://doi.org/10.1023/A:1010933404324
5. Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 785–794 (2016). https://doi.org/10.1145/2939672.2939785
6. Engelen, G., Rimmer, V., Joosen, W.: Troubleshooting an intrusion detection dataset: The CICIDS2017 case study. In: Proceedings of the IEEE Security and Privacy Workshops (SPW) (2021). https://doi.org/10.1109/SPW53761.2021.00009
7. Garcia, L.A.C., de Souto, M.C.P.: Global optimisation methods for choosing the connectivity pattern of N-tuple classifiers. In: Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN). pp. 2263–2266. Budapest, Hungary (2004). https://doi.org/10.1109/IJCNN.2004.1380975, https://doi.org/10.1109/IJCNN.2004.1380975
8. Kull, M., Silva Filho, T., Flach, P.: Beta calibration: a well-founded and easily implemented improvement on logistic calibration for binary classifiers. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS). vol. 54, pp. 623–631. PMLR (2017)
9. Moustafa, N., Slay, J.: UNSW-NB15: A comprehensive data set for network intrusion detection systems. Military Communications and Information Systems Conference (MilCIS) (2015). https://doi.org/10.1109/MilCIS.2015.7348942, https://doi.org/10.1109/MilCIS.2015.7348942
10. Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., Ghorbani, A.A.: CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. Sensors **23**(13) (2023). https://doi.org/10.3390/s23135941

11. Open Information Security Foundation: Suricata: Open source ids/ips/nsm engine. https://suricata.io/, accessed 2026-04-16
12. Platt, J.C.: Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In: Advances in Large Margin Classifiers. pp. 61–74. MIT Press (1999)
13. Roesch, M.: Snort: Lightweight intrusion detection for networks. In: Proceedings of the 13th USENIX Conference on System Administration (LISA '99). pp. 229–238. Seattle, WA, USA (1999)
14. Santiago, L.A.Q., Verona, L.T., Rangel, F.d.O., Firmino, F.M., Lima, P.M.V., França, F.M.G.: FPGA implementation of weightless neural networks for network intrusion detection. In: Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN) (2020). https://doi.org/10.1109/IJCNN48605.2020.9207591, https://doi.org/10.1109/IJCNN48605.2020.9207591
15. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP) (2018). https://doi.org/10.5220/0006639801080116, https://doi.org/10.5220/0006639801080116
16. Susskind, Z., Arora, A., Bacellar, A.T.L., Dutra, D.L.C., Miranda, I.D.S., Breternitz Jr., M., Lima, P.M.V., França, F.M.G., John, L.K.: FWIW: Weightless neural network inference at ultra-low cost. In: Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA) (2023). https://doi.org/10.1145/3543622.3573140, https://doi.org/10.1145/3543622.3573140
17. Susskind, Z., Arora, A., John, L.K., John, E.B., Lima, P.M.V., França, F.M.G.: BTHOWeN – binary thermometer-encoded hashed optimized weightless neural networks. In: Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT) (2022). https://doi.org/10.1145/3559009.3569680, https://doi.org/10.1145/3559009.3569680
18. Teresa Bernarda Ludermir, André de Carvalho, A.P.B., de Souto, M.C.P.: Weightless neural models: A review of current and past works. In: Neural Computing Surveys 2 - ICSI Berkeley. pp. 41–61. Berkeley, USA (1999), https://www.cin.ufpe.br/~tbl/artigos/vol2_2-ncs-1999.pdf
19. Umuroglu, Y., Akhauri, Y., Fraser, N.J., Blott, M.: LogicNets: Co-designed neural networks and circuits for extreme-throughput applications. In: Proceedings of the International Conference on Field-Programmable Logic and Applications (FPL) (2020). https://doi.org/10.1109/FPL50879.2020.00065, https://doi.org/10.1109/FPL50879.2020.00065
20. Zhao, Z., Sadok, H., Atre, N., Hoe, J.C., Kim, V.S., Gibbons, P.B.: Achieving 100Gbps intrusion prevention on a single server. In: Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI) (2020), https://www.usenix.org/conference/osdi20/presentation/zhao-zhipeng
21. Zoghi, Z., Serpen, G.: Building an intrusion detection system on UNSW-NB15: Reducing the margin of error to deal with data overlap and imbalance. Concurrency and Computation: Practice and Experience (2024). https://doi.org/10.1002/cpe.8242, https://onlinelibrary.wiley.com/doi/full/10.1002/cpe.8242

A Full Threshold Mode Breakdown

Tables 8–10 report all seven threshold calibration modes for each dataset. The main body (Tables 1, 3, 4) shows only the three key modes (train_cal, fixed_05, val_cal).

Table 8. UNSW-NB15 temporal split, all threshold modes (101 runs, GA Neurons best-fitness genome, balanced fitness weights).

Threshold Mode	F1-macro (%)	FPR (%)	Accuracy (%)
Train-cal	87.02 ± 2.58	13.76 ± 4.63	87.16 ± 2.53
Fixed 0.5	83.99 ± 7.26	17.99 ± 15.15	84.68 ± 6.17
Platt	83.50 ± 4.58	23.31 ± 15.07	84.26 ± 4.07
Beta	82.07 ± 5.42	28.59 ± 16.80	83.18 ± 4.50
Empirical	75.91 ± 11.29	44.24 ± 20.28	78.86 ± 7.78
Empirical cumul.	82.02 ± 6.09	29.36 ± 16.89	83.20 ± 4.96
Validation	87.34 ± 2.12	5.43 ± 4.02	87.37 ± 2.10

Table 9. CICIDS2017 random split, all threshold modes (120 runs, 16-bit thermometer, GA Neurons best-fitness genome).

Threshold Mode	F1-macro (%)	FPR (%)	Accuracy (%)
Train-cal	99.28 ± 0.52	0.24 ± 0.27	99.55 ± 0.33
Fixed 0.5	99.09 ± 0.63	0.45 ± 0.29	99.42 ± 0.40
Platt	99.18 ± 0.52	0.34 ± 0.21	99.48 ± 0.33
Beta	99.16 ± 0.69	0.30 ± 0.19	99.47 ± 0.42
Empirical	98.41 ± 1.31	1.14 ± 1.10	98.96 ± 0.91
Empirical cumul.	99.07 ± 1.52	0.26 ± 0.16	99.43 ± 0.82
Validation	99.30 ± 0.48	0.22 ± 0.12	99.56 ± 0.30

Table 10. CIC-IoT-2023 random split (1.3M subsample), all threshold modes (56 runs, 8-bit thermometer, GA Neurons best-fitness genome).

Threshold Mode	F1-macro (%)	FPR (%)	Accuracy (%)
Train-cal	80.06 ± 0.50	4.32 ± 0.46	86.85 ± 0.44
Fixed 0.5	81.60 ± 0.40	9.11 ± 0.93	88.55 ± 0.34
Platt	81.38 ± 0.52	33.12 ± 3.25	90.72 ± 0.18
Beta	81.10 ± 0.67	34.78 ± 3.55	90.71 ± 0.15
Empirical	67.96 ± 14.22	64.87 ± 24.03	88.69 ± 2.37
Empirical cumul.	82.05 ± 0.81	23.03 ± 8.19	90.21 ± 0.42
Validation	79.88 ± 0.50	3.97 ± 0.52	86.65 ± 0.44